Distributed Systems

CS244B Course Notes

David R. Cheriton
Computer Science Department
Stanford University
Stanford, CA 94305

March 25, 2011

Copyright 1989-2011 David R. Cheriton, All rights reserved.

Contents

1 Overview **17**

1.1 Views of a Distributed System 18

1.1.1 Shared State View of Distributed Systems 18

1.1.2 Object View of Distributed Systems 22

1.1.3 Protocol View of Distributed Systems 23

1.1.4 Focusing on Shared State vs. Communication 24

1.1.5 Trusting Shared State 25

1.2 Why Distributed Systems? 26

1.2.1 Composition 26

1.2.2 Decomposition 27

1.2.3 The Challenge 27

1.3 Distributed System Building Blocks 28

1.3.1 Computer Nodes of a System 28

1.3.2 Interconnecting Network 29

1.3.3 Distributed System Software 32

1.4 Example Systems 39

1.4.1 Software Development System 39

1.4.2 Distributed Banking System 41

1.4.3 Embedded Real-time Distributed Systems 44

1.4.4 HPC Cluster 44

1.4.5 Server Cluster 46

1.4.6 Strategic Defense System 47

1.4.7 Distributed Interactive Simulation (DIS) 48

1.4.8 World-Wide Web 50
1.4.9 Service-oriented Architectures 51
1.4.10 Unanticipated Distributed Applications 52
1.5 Overview of Topics 53
1.6 Concluding Remarks 56

2 Distributed Shared Memory 58
2.1 Why DSM? 58
2.2 Shared Memory Consistency 60
2.3 Basic DSM Consistency Protocols 61
2.3.1 DSM Directory: Page Meta-information 63
2.3.2 DSM versus Shared Memory Multiprocessor 64
2.4 DSM Performance and Program Behavior 65
2.4.1 Fine Grain Consistency 67
2.4.2 Program Restructuring 68
2.4.3 Logical and Physical Contention Control 69
2.4.4 Heterogeneity 69
2.4.5 Fault-tolerance and the Tight Coupling of DSM 70
2.4.6 Security 70
2.5 Specialized Consistency Protocols 72
2.5.1 Munin and Treadmarks 72
2.5.2 Memory-based Messaging 73
2.6 Specializing the Memory Interface 74
2.6.1 Linda 74
2.6.2 Javaspaces, OpenSpaces and Ramcloud 77
2.7 The Spectrum of State Sharing Techniques 77
2.7.1 Direct Communication Techniques 77
2.7.2 Consistent Shared Memory 80
2.7.3 Application-Specific Shared Memory 81
2.8 Concluding Remarks 84

3 Structuring of Distributed Systems **87**
3.1 History . . . 87
3.1.1 Messages: A Communication Abstraction . . . 88
3.1.2 Distributed File Systems: The File Abstraction . . . 90
3.1.3 UDP/TCP: Socket Abstraction . . . 92
3.1.4 Distributed Shared Memory (DSM) . . . 93
3.2 Object-oriented RPC Approach . . . 94
3.2.1 OO RPC Implementation . . . 96
3.2.2 Using (OO) RPC to Define a Network Service . . . 99
3.2.3 Example RPC Systems . . . 100
3.2.4 Advantages . . . 105
3.3 Problems with RPC . . . 106
3.3.1 Performance . . . 106
3.3.2 Semantics . . . 107
3.3.3 Language/Procedural Restrictions and Complications . . . 109
3.4 Alternatives . . . 111
3.4.1 Smart Proxies . . . 111
3.4.2 REST . . . 116
3.4.3 Considering the Web . . . 117
3.4.4 Attributed-oriented Stateful Proxy . . . 117
3.5 OO Interface Design . . . 120
3.5.1 Object References . . . 121
3.5.2 Resource Pools and Managers . . . 122
3.5.3 Recursive Structuring . . . 123
3.5.4 Example Interface: UIO . . . 125
3.6 Concluding Remarks . . . 132

4 Clocks: Real, Virtual and Logical **136**
4.1 Real-time Clock Synchronization . . . 137
4.1.1 NTP . . . 138
4.1.2 NTP Management . . . 141
4.1.3 Cheriton's Contrarian View on NTP Management . . . 142

4.1.4 Other Work 144
4.2 Virtual Time Clocks 144
4.3 Logical Clocks 146
4.3.1 Communication-Level (Logical) Clocks 150
4.3.2 State Machine Approach 151
4.3.3 Mutual Exclusion 154
4.4 Synchronized Time, Stateful Proxies and Sufficient Consistency . . . 156
4.5 Concluding Remarks 157
4.6 Readings 158
4.7 References 158

5 Directories and Naming 160
5.1 Naming 'Theory' 161
5.1.1 Proper Names, Common Names and Descriptions 162
5.1.2 Names versus Types 164
5.1.3 Pure Names and Impure Names 165
5.1.4 Structured Names 165
5.2 Naming and References in Object-oriented Systems 166
5.2.1 External versus Internal Object Names 167
5.2.2 Conventional Naming Thinking vs. Cheriton's Approach . . . 167
5.3 Internal Naming 169
5.3.1 Case Study: IP Addresses 169
5.3.2 Case Study: V Process and Object Identifiers 171
5.3.3 UUIDs Considered Harmful 172
5.3.4 Proper Use of Internal Names 174
5.4 External Naming Implementation 175
5.4.1 Centralized versus Decentralized Naming Authority 176
5.4.2 Global Directory Architecture 177
5.4.3 Lampson's Global Name Service 178
5.4.4 Epidemic Approach 180
5.4.5 Naming in the V-System 181
5.4.6 Administrational Directories 184

5.4.7 Security with Decentralized Naming 185
5.5 URLs vs. Academic Name Theory 186
5.6 Location-based Naming and Identification 187
5.7 Directory Services versus Databases 190
5.7.1 UDDI 190
5.8 Resource Discovery and Search 191
5.9 Concluding Remarks 193

6 Accounts and Authentication 195
6.1 Encryption Technology Base 197
6.1.1 Shared vs. Public Key Encryption 198
6.1.2 Logic of Authentication 199
6.1.3 Encryption vs. System-level Security 200
6.2 Key Management 201
6.2.1 Key Distribution 202
6.2.2 Trusted Third Party 203
6.2.3 Optimizing the Authentication Service 203
6.3 Authentication Schemes 204
6.3.1 Example Authentication Service: Kerberos 205
6.3.2 X.509 206
6.4 Digital Signatures 206
6.5 Certificate-Based Authentication 207
6.5.1 Revocation 208
6.5.2 Standards and Industry Efforts 208
6.5.3 PKI Certificates Considered Harmful 209
6.5.4 More Certificate Challenges 210
6.5.5 Certification versus Authorization 211
6.6 Large-Scale Authentication 211
6.6.1 Similarities to the Time Service Approach 212
6.6.2 Scalability and the “Small-World” Model 213
6.7 Naming vs. Authentication 214
6.8 Open Security, Physical Security and Availability 214

6.9 Trust and Reputation 218
6.10 Accounting 219
6.10.1 Amoeba Banker 220
6.10.2 Electronic Wallets 220
6.11 Concluding Remarks 221

7 Transactions, Agreement and Reconciliation 223
7.1 Transaction as a Distributed Object 223
7.2 Advantages of Transactions 224
7.3 Commit Protocols 226
7.3.1 The 1-phase Commit Protocol 226
7.3.2 The 2-Phase Commit Protocol 226
7.3.3 Non-blocking Commit Protocols 227
7.3.4 Byzantine Commit Protocols 228
7.4 OOTP — Example Protocol 228
7.4.1 Transaction Identification 229
7.4.2 Transaction Operations 230
7.4.3 Normal Commit Protocol 231
7.4.4 Immediate Commit Protocol 232
7.4.5 Fault-Tolerant Commit 232
7.4.6 Byzantine Mode 234
7.4.7 Nested Transaction 234
7.4.8 Implementation 234
7.4.9 Other Uses 237
7.4.10 Performance Evaluation 237
7.5 Other Example Distributed Transaction Implementations 238
7.5.1 Camelot Transaction Manager 238
7.5.2 MTS: Microsoft Transaction Server 239
7.5.3 QuickSilver 240
7.5.4 ISO CCR 240
7.5.5 Atomic Actions 241
7.5.6 PAXOS Algorithm/Protocol 241

7.6 Long-lived Transactions 242
7.7 Replicated Data Management 243
7.7.1 Replicated Group Membership 244
7.7.2 Detecting Failures 244
7.7.3 Network Partitions 245
7.7.4 Alternative Replication Schemes 246
7.8 Reconsidering the Conventional Atomic Transaction Approach . . . 247
7.8.1 Round-trip and Commit and Byzantine Protocols 248
7.8.2 Transaction Isolation Levels 249
7.9 Asynchronous Stateful Proxy Approach and Transactions 250
7.10 Knowledge-based Reasoning about Distributed Systems 253
7.11 Approximate versus Exact Agreement 255
7.12 Conclusions 256
7.13 Readings 257
7.14 References 257

8 Future Directions and Issues 260
8.1 Distributed Systems as File Transfer 260
8.2 Distributed Systems as Shared Object Models 262
8.3 The New Centralization 263
8.4 "Consumer" Distributed Systems 265
8.5 Computing Clusters 267
8.6 Corporate Distributed Systems 268
8.7 Specialized Embedded Distributed Systems 270
8.8 "Stratified" Distributed Systems 272
8.9 Ubiquitous Distributed Computing 273
8.9.1 Mobile and Cellular Phone Computing 274
8.9.2 Peer-to-Peer Systems? 275
8.10 The Next Generation of Design? 276
8.11 Concluding Remarks 277
8.12 References 278

9 Distributed/Network File Systems **279**

9.1 I/O Protocols — The Interface 279

9.1.1 Example: The UIO Interface 280

9.1.2 NFS: A second example 281

9.1.3 CIFS: Common Internet File System 282

9.1.4 FTAM: ISO Does File Access 282

9.1.5 Locus 284

9.2 File Caching 284

9.2.1 Why File Caching 285

9.2.2 Issues 285

9.2.3 File Usage Characteristics 286

9.2.4 Example Approaches 286

9.2.5 Coda 288

9.3 Scalability/Managability 289

9.4 Caching Structured Data and Distributed Databases 290

9.5 WWW "Push" Model 291

9.6 Concluding Remarks 291

10 Internet Content Distribution **293**

10.1 Spectrum of Types of Content 294

10.2 Real-time Video Delivery and IP Multicast 295

10.3 Web Caching and Replication 296

10.3.1 Web Traffic 296

10.3.2 Forward Proxies (AKA Caches) 298

10.3.3 Reverse Proxy and Akamai 299

10.4 Cookies vs. Statelessness 300

10.5 Content Transformations 300

10.6 Issues 300

10.6.1 Cache Consistency 301

10.6.2 Reliable Multicast 301

10.6.3 Mapping the Internet 302

10.6.4 Load Partitioning 303

10.6.5 Directory Overhead 303
10.6.6 SLAs and Performance 304
10.6.7 Improving Caching Performance 304
10.6.8 Caching and Proxying Policies 305
10.6.9 Multi-Cache Organization 306
10.6.10 Edge Computation 306
10.6.11 Content Architecture 307
10.7 Related Areas 307
10.7.1 Email as Content Distribution 307
10.7.2 Napster 308
10.8 Concluding Remarks 308
10.9 Readings 309
10.10 References 309

11 Distributed Scheduler: Program Execution and Management 312
11.1 Scheduler Implementation 313
11.1.1 Scheduler Structure 313
11.1.2 Program Execution Mechanism 316
11.2 Migration 318
11.2.1 The V Migration Facility 319
11.2.2 Sprite Process Migration 320
11.2.3 Unprotected Operating Systems 321
11.3 Scheduling Policy 321
11.3.1 Example Scheduling Algorithm 322
11.3.2 Parallel Program Scheduling 323
11.4 Fault-tolerance and Recovery 325
11.4.1 Replicated Processes 325
11.4.2 Replicated State 326
11.4.3 Replicated Communication 327
11.5 Scheduling in Web Server Clusters 328
11.6 Concluding Remarks 329
11.6.1 References 330

12 Semi-Autonomous Distributed Systems **331**
12.1 Situational Awareness 331
12.2 Motivations and Challenges 332
12.2.1 Example: Alaskan Bush Pilots and Air Traffic Control 332
12.2.2 Disadvantages 334
12.3 Implicit versus Explicit Communication 334
12.4 Real-world Environments 334
12.5 Example Systems 335
12.5.1 Internet Autonomy 335
12.5.2 Train Systems 335
12.5.3 Air Traffic Control 335
12.5.4 Financial System 336
12.5.5 Intelligence Systems 336
12.6 The Challenge of Safety Mechanisms 337
12.7 Comparison to Distributed Interactive Simulation 337
12.8 Comparison to Peer-to-Peer 337
12.9 Trust 338
12.10 Evolution 338
12.11 Competitive Systems 339
12.11.1 Simulation-based Evaluation 340
12.11.2 Versus Game Theory 340
12.12 Concluding Remarks 340

13 Protocol Architecture and Presentation Protocols **342**
13.1 Protocol Architecture 343
13.1.1 End-to-End Argument 346
13.1.2 Open Architectures 346
13.2 Presentation Layer 347
13.3 Presentation Protocol Design 348
13.4 ISO 8825 BER/X.409: Explicit Specification 349
13.4.1 Types 350
13.4.2 Length 351

13.4.3 Implementation Approaches 351
13.4.4 Summary of BER/ASN.1 351
13.5 XDR: Implicit Specification 352
13.6 ASCII Rules! 353
13.7 Remote Procedure Call (Re)Presentation 354
13.7.1 Procedure Identification (Re)Presentation) 354
13.7.2 Complex Data Parameters 355
13.7.3 Reliability and Failure Semantics 356
13.7.4 Secure RPC 357
13.7.5 Call Streaming 358
13.8 Instruction Stream Model 360
13.9 Concluding Remarks 361
13.10 Readings 361
13.11 Informal Bibliography 361

14 Communication Support **363**
14.1 Conversational Communication 363
14.1.1 Naming 364
14.1.2 Presentation 364
14.1.3 Transport 364
14.1.4 Similar Protocols 366
14.1.5 Issues 366
14.2 Transactional Communication 367
14.2.1 Example: V Message Transaction and Stubs 367
14.2.2 Datagrams and Idempotency 368
14.2.3 Multicast 368
14.2.4 VMTP: Transport Protocol 370
14.2.5 Recursive Communication Architectures 371
14.2.6 Critique of VMTP 371
14.3 Dissemination-oriented Communication 372
14.3.1 Applications and Motivations 372
14.3.2 Comparison to Past Models 373

14.3.3 Dissemination Support 374
14.3.4 Critique 374
14.4 Fault Tolerance 375
14.5 Kernel Communication Primitives 376
14.6 Concluding Remarks 376
14.7 Readings 377
14.8 Informal Bibliography 377

List of Figures

1.1 Distributed System 17
1.2 Shared State View of a Distributed Application 19
1.3 Distributed System/Application Structuring 20
1.4 Software Development System 40
1.5 PC/Workstation-based Banking System 42
1.6 Mainframe-based Banking System 42
1.7 Workstation/Server-based Banking System 43
1.8 Wargaming: Distributed Interactive Simulation 48

2.1 Inconsistent Ordering of Reads and Writes 60
2.2 State Transition Diagram for a DSM Page 62
2.3 Spectrum of State Sharing Techniques 78
2.4 Direct Communication of Shared State 79
2.5 Indirect Communication Through Shared State 80
2.6 Cached Indirect Communication Through Shared State 80
2.7 Amaze Multiplay Game 83

3.1 Object-oriented Remote Procedure Call 97
3.2 Kernel as Base for Recursive Structure 104

4.1 NTP Message Exchange 139
4.2 Distributed Stock Trading 147
4.3 File Cache Consistency Problem 149

5.1 Old V Process Identifier Structure 171

7.1 Prepare-to-commit 231
7.2 Commit Phase 231
7.3 Immediate Commit Protocol 232
7.4 Fault-tolerant Commit Protocol 233

8.1 Progress as Moving up the Layers 261

11.1 V Program Execution 317

13.1 Protocol Architecture 342
13.2 Protocol and Interface Layers 344

14.1 Send-Receive-Reply Message Transaction 367
14.2 Hand-generated V stub routine 369

List of Tables

10.1 Web Sites and Traffic 297

14.1 Telnet Presentation Protocol 365

Chapter 1

Overview

A *distributed system* is a set of communicating autonomous entities that are functioning together to achieve a common goal[1]. More specifically, for our purposes, a distributed system is a collection of *computer nodes*, connected by a network, running software that makes it function as one *system*, as depicted in Fig. 1.1.

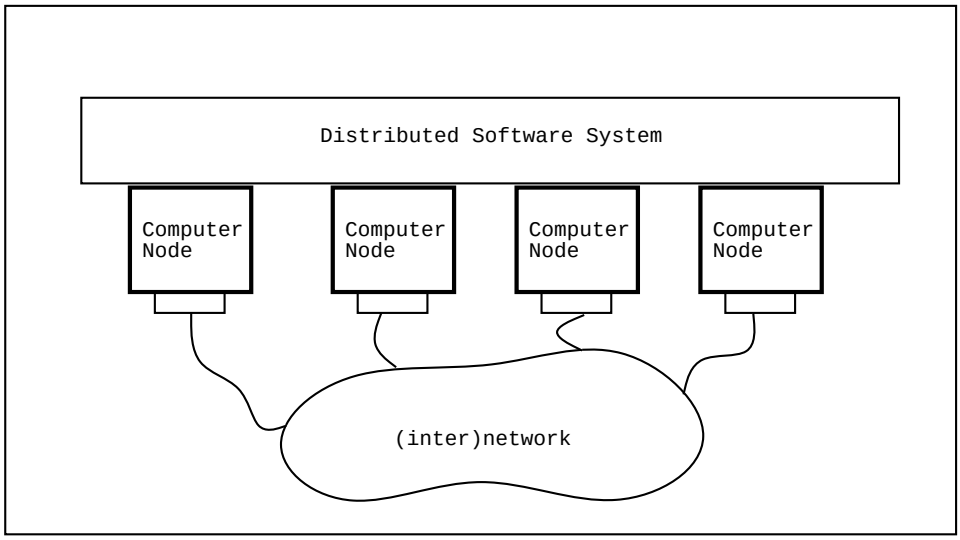

Figure 1.1: Distributed System

There are many different types of distributed systems. For example, intelligent

[1]The dictionary definition of a *system* is a set of things related, arranged, organized, etc. so as to form a unity or functioning whole.

terminals and protocols (such as X-terminals, Sunray's or VDI) attached to a mainframe or server is considered a distributed system by some[2]. However, the limited functionality of the "terminals" generally makes this type of distributed system quite similar to a conventional mainframe approach with terminals. A more important type of distributed system arises with a cluster of nodes with significant local processing, memory and storage capacity forming a multi-computer, or computing cluster. Rather than a central machine just telling a bunch of dumb terminals what to do and taking simple inputs, there is the matter of coordinating a number of nodes with significant local processing and state. However, even in this context, there is normally a central control that monitors and controls the rest of the nodes. Consequently, a distributed system is normally structured according to a *hierarchy* of nodes, from a top-level operator/controller node down to limited function *customer-premises equipment(*CPE), to use a classic telco term. For example, a search cluster being used from web browsers is structured as a top-level facility for monitoring and managing search and indexing nodes at the next level that interface to web servers that interface to browser nodes (as CPE in this case) at the next level. Yet, with these fancy examples, let's not forget a modest, but common and still challenging case of a set of processes running in disjoint address spaces within a single box connected by some form of interprocess communication. This is also a distributed system, given that the processes need to communicate without the benefit of sharing a common address space. Here, the nodes and the communication are "virtual". In that vein, most of the issues of a distributed systems arise as soon as an application goes to multiple processes, given the application can then no longer rely on a shared address space for sharing state between its modules.

1.1 Views of a Distributed System

There are several different high-level *views* one can take of a distributed system, namely the *shared state view* , the *object view* and the *protocol view* , as elaborated below.

1.1.1 Shared State View of Distributed Systems

A distributed system is a single "system" because it provides *shared state* between its participating nodes, as illustrated in Fig.1.2. Here, *shared state* is state that is synchronized to the degree required by its use between various nodes of the system. For example, a distributed multiplayer game is a single game even though separate copies are running on separate workstations because the copies share a common

[2]The so-called *virtual desktop* has become a significant part of the new centralization of corporations to the data center.

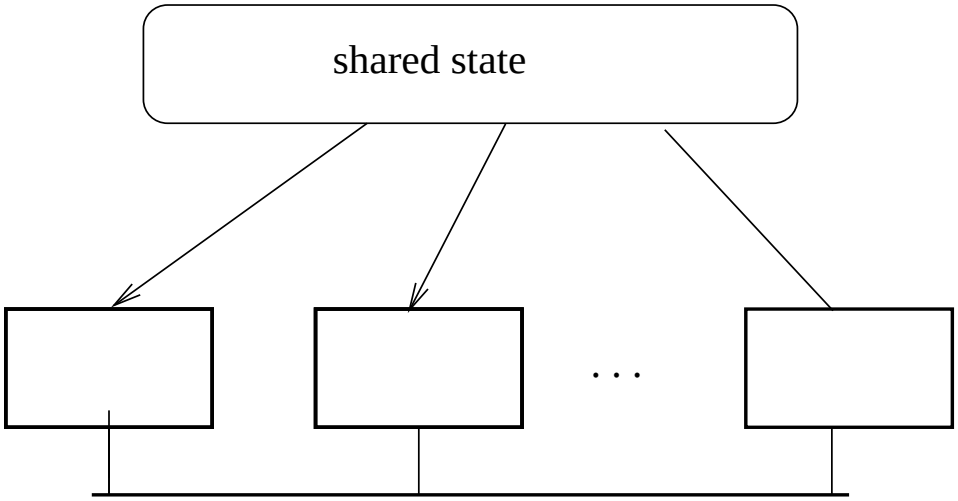

Figure 1.2: Shared State View of a Distributed Application

game state. This state is synchronized so players see the same state of the game at the same time. In reality, there are often separate memory copies of the state on the different nodes. However, protocols between the nodes are used to keep the separate copies *consistent*. It is generally too expensive to keep the state strictly consistent so the system only maintains *sufficient consistency* , sufficient for the application/services that use the state.

We emphasize a shared-state centric view of distributed systems as well, identifying a spectrum of shared state consistency, and the associated costs and benefits. These topics are covered, starting Chapter 2.

You can think of a distributed system as structured as three layers, as depicted in Fig. 1.3. These layers are:

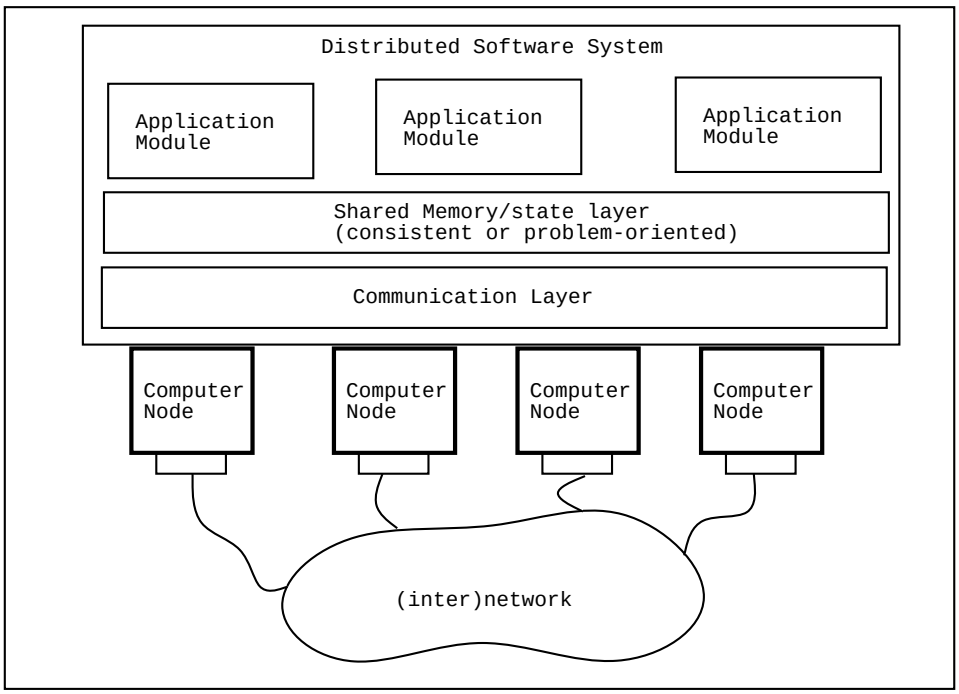

Figure 1.3: Distributed System/Application Structuring

1. application/service-specific logic layer
2. shared state layer
3. communication layer

The top layer consists of application code that operates using the shared state but is (largely) independent of the fact that the shared state is distributed. The shared state layer maintains that shared state, including implementing protocols that keep the state sufficiently consistent for the clients of this state. The communication layer provides communication support for the shared state layer, sometimes just facilities for transmitting and receiving packets with the associated buffering, queueing, demultiplexing and filtering.

For example, continuing the example of the distributed game, the application logic layer performs the moves and updates on the game state maintained in the state layer, which then reflects these moves over the network to other nodes in the game, causing their state to be updated to reflect the local player's moves.

Note: in reality, there is state at all three layers. The "shared state" is that which is logically replicated and synchronized across several nodes, as opposed to local state[3].

Considering the handling shared state, distributed systems and applications range from having very loose coupling to very tight coupling based on the coupling to shared state and its consistency. The *coupling* is based on:

1. the level of access to state: lower-level access such as remote read and write of memory produces tighter coupling where is higher-level access provides greater protection between systems and thus, looser coupling.

2. strictness of consistency: some systems require strict consistency of the shared state and thus tight coupling whereas other systems can cope with looser consistency.

3. communication performance requirements: tightly coupled systems are highly dependent on communication performance for their performance whereas loosely coupled systems are not. In particular, tightly coupled systems are often dependent on low-latency communication.

These three aspects are interdependent in the sense that lower-level access tends to require more strict consistency (because there are limited semantics to exploit) and strict consistency requires low-latency communication to minimize the cost of nodes waiting for the current version of the state or completion of the coherency protocol, allowing them to update the state.

Examples of loose coupling include Web access, telnet, FTP and e-mail services. With this loose coupling, a browser does not crash (normally) just because the

[3] A common model for modern applications is three layers corresponding to presentation, business/application logic and data access. The data access layer corresponds to our shared data layer. We ignore the presentation/application logic separation because it is not tied to distributed. We further identify the communication layer, because that is the basis for distributed operation.

web server it is accessing has crashed. Similarly, a web server generally operates independent of other browsers or servers crashing.

An example of tight coupling is a distributed parallel application running on a distributed shared memory system implemented over a compute cluster. In a tightly coupled system, a memory reference may fail because of a network or remote node failure, generally resulting in a local application or system failure.

Overall, in this view of distributed systems, you should *think (shared) state, not messages* as the central distributed systems issue, with the associated issues of what is shared, how it is shared, etc.

1.1.2 Object View of Distributed Systems

The state of a distributed system can be viewed as a set of *objects* with *interfaces* that clients access, ideally through a uniform means of referencing the objects and invoking calls on these objects independent of their implementation.

Just as with normal object-oriented programming, this structuring approach helps deal with the complex and diverse state occurring in a real-world large system, hiding the different implementations from clients behind the interfaces. However, within a distributed system, the interface also specifies application- or service-specific semantics associated with the operations on the state, allowing a distributed implementation to exploit these specific semantics to provide (just) *sufficient consistency* for these semantics. In particular, type-specific techniques can be used with different types of state, with object-oriented types invoking the appropriate means based on the typing of the state. The types can effectively tag the state according to semantics, access and consistency maintenance.

For example, a `GamePlayer` object can know that it is sufficient to periodically update copies of its state at other nodes rather than on every update, especially if using smooth animation, recognizing strict consistency is not needed. A distributed system also exploits the multiple implementations of interfaces in object-oriented programming to allow "remote implementations", often called proxy or stub implementations.

As with object-oriented programming in general, the object view of distributed systems stresses how to structure the state of the system. That is, how to partition into separate modular units that can be accessed by a well-defined interface. The distributed object world just raises the issue of modularity further. How to define this modularity so it works well to provide security, availability and performance across network boundaries.

Chapter 3 discusses the object-oriented structuring of distributed systems in detail.

In the OOP view, an *interface* is a protection barrier, separating the client from the implementation. In distributed systems, that barrier is implemented across a

wire in terms of one or more *protocols*, leading to the protocol view of distributed systems.

1.1.3 Protocol View of Distributed Systems

A distributed system can be viewed as defined by the protocols it uses, i.e. the syntax, semantics and timing of packets (or messages) it sends over the network to effect operation as a system. These protocols specify the hard part of distributed systems design, namely how the state and functionality is factored between the nodes and how they interact, i.e. how the state and functionality are factored. Once the protocols are determined, implementing the distributed system is a small matter of programming; and the latter is programming "in-the-small" relatively speaking compared to the size of a serious distributed system.

A *protocol* is specified as the rules and conventions for communication between peer levels, including protocol syntax, semantics and timing. The sender and receiver must speak/understand a common language to transmit "meaning". The timing aspect arises both in terms of time-outs, typically at the transport layer, as well as in the (logical time) ordering of operations. Timing is also often the primary means to detect failure both because failure to respond within a given time is a failure and also because timeout if often the only means to determine that another node has failed.

Any implementation of these protocols is able to participate in the system, interoperating with completely independent implementations, because it uses the same protocols. For example, you can develop a new web server that works with existing browsers and web caches independent of how you write the software, provided that you provide a compliant implementation of IP/TCP/HTTP (and all the other requisite protocols).

The protocol design of a distributed system has a significant impact on its performance, in particular its scalability in terms of bandwidth, delay and effect of loss of communication or of any given node. For example, a system whose protocols only support a single top-level node with no fail-over provision with a tight-coupling to this top-level node is intrinsically limited in its ability to scale its performance or fault-tolerance. Also, a system that uses a protocol whose bandwidth cost grows quadratically in the number of nodes in the system has a scaling problem too. Furthermore, a protocol that imposes synchronous round-trip times exposes the system to growing delay and unpredictability with scale, generally meaning that it cannot scale and preserve reasonable performance.

The specification of protocols is still rather primitive. Looking at current Internet protocol specifications, they are typically specified in primitive protocol data unit (PDU)s at the bit (assembly language) level with semantics/procedural behavior and

timing in informal natural language (monkey language). There has been considerable effort in formal languages for specifying protocols, but primarily for protocol verification, Estelle being one example. However, there is a significant gulf between these languages and how protocols are specified and implemented in practice. This is an area that calls out for improvement.

This book thus emphasizes the design-in-the-large aspects of distributed systems, namely the protocol design, rather than the local implementation techniques. For example, we emphasize how one communicates with a file server rather than how the file server is implemented, except for aspects of the communication facilities. The internal implementation of a file server is basically just a file system, for which the implementation techniques are the same as for conventional file systems (In fact, most file servers are built on existing conventional file systems.).

That is, *Think protocols, not software.*

Unifying the story with these three views, the state of a (distributed) system is structured and encapsulated in objects, with the (object-oriented) interfaces specifying the logical access by clients to each of these modular units of state, and high-level protocols determined from these interfaces specifying the communication that takes place over the wire.

1.1.4 Focusing on Shared State vs. Communication

The on-going mistake with distributed systems is focusing on the communication layer rather than a shared state layer. I argue that the shared state focus is necessary and sufficient to address all distributed system issues, whereas a communication layer focus is neither necessary nor sufficient.

First of all, the shared state focus is sufficient because, if you could assume that the shared state required by an application is available and consistent in each node of the system, you can simply write procedures to read and write this state, oblivious to the application being distributed and the application should work, assuming correct local programming.

It is necessary by the following argument. We require a distributed system or application to work correctly according to some requirements. If it does not work correctly, at least one node must be doing something wrong relative to the other nodes. A node is just a state machine, executing procedures/state transition functions on its state. If it did do something wrong, it must be because of an error in its procedure, its local state or the shared state (ignoring hardware errors in this discussion). (The former two are just standard software engineering issues and have nothing to do with being distributed.) Thus, to ensure correct operation of a distributed system, it is necessary to get the shared state layer right.

The communication or message layer is not sufficient to consider because messages have no effect (other than consumption of bandwidth) unless they modify the state of a node. Moreover, the execution of a node is never just dependent on the contents of a message; it is also dependent on the node's internal state. You can add extra state to a communication layer service but that does not constitute all the state associated with application so can never be the whole story.

By this reasoning, this text focuses on shared state and state consistency, derives requirements for communication based on these, and provides discussion of mistakes that focus too much on the communication layer. You might regard this as clarification of the *end-to-end argument*, recognizing the state at the true ends.

1.1.5 Trusting Shared State

Given that the correct behavior of a distributed systems depends on the shared state, it is necessary to be able to "trust" this shared state. What does this mean? Well, informally, it means that other nodes in the system are not changing the shared state in ways that are incorrect or inconsistent or not authorized. For example, in a distributed game, a player should not be able to update the game state to indicate that it made a move that was not legal in the game, or gain points to which it was not entitled, or change the position or score of some other player.

Trust may seem like a pretty mushy term to use in a technica course. However, you can define *trust* in an agent A as: expectation of good future behavior by an agent A. This trust is based on:

1. observation of good behavior by A in the past over some period of time,
2. attestation by some trusted agent B as to A's trustworthiness.
3. absence of complaints about A

Thus, a basic requirement for trust is trustworthy identification over time. I.e. how can we talk about agent A unless we are confident that A identifies the same agent over time. Clearly, attestation also depends on identification. Another requirement is *observability*, so the good behavior can be observed and any bad behavior can be detected.

In a distributed computer system, activity or events are "observed" by communication of messages. Thus, maximum observability is achieved by multicasting all messages to every participating node. This is effectively how air traffic control works (augmented with visual observability). That is, all communication between each pilot and the air traffic controllers is broadcast within the local air space, so every participant can "observe" everyone else.

On a small scale, it may be feasible to rely on every node being trustworthy based on physical security. For example, we might trust the database server because it is in a physically secure location managed by a trusted DBA. Similarly, on a small scale, it may be feasible to multicast all communication to all participating nodes. However, at larger scales, neither is feasible. Physical security cannot be ensured across larger regions at reasonable cost. Moreover, multicasting tends to lead to a quadratic growth in packet reception traffic at each node. Moreover, it is not even feasible to "trust" that traffic is actually multicast everywhere, given the network can be compromised too.

As we shall discuss, the conventional wisdom of relying on fancy "security" mechanisms such as public key certificates is really putting one's trust in a *secret*, and secrecy also does not scale.

Thus, a key issue for distributed systems is providing trustworthy shared state. Subsequent chapters consider trust as part of various topics.

1.2 Why Distributed Systems?

Lots of reasons, but they arise in two basic ways: *composition* and *decomposition*, motivated by the desire to share information and share (hardware) resources, sometimes software resources in the form of restrictively licensed software.

1.2.1 Composition

Composition — of existing systems to form a larger distributed system.

E.g., electronic mail system on the ARPAnet, interconnects the e-mail systems on different machines which existed prior to the ARPAnet.

Often, the entities are given and communication chosen to meet established requirements.

information sharing — access to remote databases, as with the Web.

Versus independent isolated computers.

Note: computers as information processors: interconnect to avoid humans as the middlemen (persons).

E.g., human transcribes computer-generated forms (such as bank statements and payroll statements) for input to income tax computer, i.e. data capture.

Instead, let's computerize a human system by taking out the (human) middleman and using networks to directly connect computers. It takes computers, network and distributed systems protocols and software to accomplish. However, it also introduces issues of security, accountability, etc. with automatic flow of information.

These days, a key activity is connecting legacy corporate systems into the web, building a distributed system in that form, e.g. using so-called *web services*.

Considering *resource sharing*, an existing node might be composed into a (distributed) system to allow access to auxiliary processing for load sharing, archival storage and specialized or proprietary program services. e.g., company sells geological survey analysis services over the Internet. Proprietary web services are another example.

1.2.2 Decomposition

Decomposition: of single entity or system into modular components. with interconnection mechanism, for reliability, modular design, etc. E.g. compute cluster, embedded system (such as switch or autonomous vehicle).

Usually entities designed to match the characteristics of the communication facility.

Why decomposition:

1. Cost-effective commodity components — use (commodity) PC technology as the base for multi-user system rather than specialized, limited market multiprocessors.
2. Reliability — graceful degradation with replicated components (also survivability and controlled *fate sharing* with separation). In some cases, you can use separate nodes based on functionality to ensure failure and resource isolation between different functions.
3. Extensibility — just add more small pieces rather than replacing the one big one.
4. Security — physically separate except for 'thin wire', allowing restricted and therefore more controllable interaction, i.e. as compared to a shared memory.
5. Space — PCs in offices versus dedicated machine rooms with large mainframes.
6. Better performance guarantees — no loss of response in mid-afternoon. For real-time, no interference in response if on a separate processor.

1.2.3 The Challenge

The design of distributed systems is harder because:

- more complex and general — e.g. K file systems versus one, and more modular — communication boundaries, with performance issues.

- more complex failure modes — always partly up and partly down, and never completely up or down.
- very large-scale design with evolution over long times and never down or fully recompiled.

In this sense, a distributed system is at best a necessary evil, *evil* because of the extra complexity. The question is whether it is necessary and in what form. We next discuss several examples of distributed systems and applications.

1.3 Distributed System Building Blocks

There are three major building block components in a distributed system: computer nodes, interconnecting network and distributed software system. The nature, costs and evolving directions (from enabling technologies) of these dictate a lot about the structure of distributed systems.

1.3.1 Computer Nodes of a System

What constitutes a 'computer node'.

1. processor
2. memory
3. network interface.

The *holy trinity* of processing, storage and communication. There may be other other peripherals such as disks, but they can be viewed as services over a network or a special case of local storage or memory.

A key trend in this area is the use of low-cost commodity blades which is moving to multi-core CPUs, memory in the many gigabytes and interconnect speeds moving to 10 Gbps. However, we do not expect to see as much increase in performance per core because of the limits the technology is running into here, as well as power requirements. That is, more performance has to be achieve with parallelism. Software problems have already pushed industry to partition services onto separate nodes (to provide failure-independence), and more recently to use separate virtual nodes using virtual machine technology, the latter reducing capital cost as well as power requirements.

Another trend is the falling cost of DRAM. It is not feasible to put many gigabytes of memory on a server blade (ignoring software issues). In particular, a server can hold as much data in memory as an entire database did just a few years ago.

Also, even with the increase in network performance, it is not clear that it is keeping pace with processor performance. For example, a 1 Mhz processor in 1986 might be connected to a 10 Mbps Ethernet where as a 3 GHz processor now is likely connected to a 1 Gbps connection (or less). Thus, we have gone from 10 to 1 ratio of processor speed to bandwidth to a .3 to 1 ratio. With the expected performance increase in processors in the next year or so, even moving to 10 GE is not returning us to the ratio we had in 1986.

1.3.2 Interconnecting Network

The (interconnecting) network is the means to implement shared state between nodes, and thus its properties significantly influence what is feasible and how it is structured.

For this course, we assume network interface providing unreliable datagram delivery, e.g., Ethernet. A virtual circuit interface is also possible, given that introduces relatively few differences other than imposing connection management overhead and complexity. Implementation of networks is not covered – see the networks course.

Similarly, a network connection can be wireless, allowing a node to be *mobile*. However, mobility and/or wireless does not provide a useful distinction in distributed systems because the software problem does not change as a result directly. It is more a matter of the properties of the network connection, and mobile is not necessarily slow or flakey.

Properties of interconnection network dictates many aspects of the system, i.e.

1. data rate
2. delay
3. reliability/availability

The ideal would be to have each of these match the characteristics of the computer node's memory system, the key mechanism for state sharing. That is,

1. high data rates, comparable to writing to memory.
2. low delay, comparable to reading from memory for roundtrip times
3. very high reliability - fails no more often than a read or write fails, which is generally assumed not to happen, i.e. we are generally willing to have the system fail if the memory bus does, because it does so infrequently.

You can generally regard the bus connecting processor to memory as a small-scale example of such an interconnecting network, so this ideal can be achieved. Thus, a typical parallel machine is a distributed system where:

- network has high-bandwidth, low delay and fast reliable broadcast, comparable to memory system performance.
- all the computer nodes are identical, statically fixed at boot time and small in number.
- failures are catastrophic to *partitions* of the machine, if not the whole thing. I.e. a node failure terminates execution of applications and usually the entire operating system as well.

How convenient!

Unfortunately, this model cannot be scaled cost-effectively. Actually, it is impossible given the inherent delays as you scale the network to the wide-area.

Delay increases with distance. For example, the speed of light delay across the continental USA is roughly 15 milliseconds one way, so the roundtrip time to do a read operation is at least 30 millisecconds, versus nanoseconds on the short local memory bus. Moreover, this is only achieved with a network that approximates a direct link from one end to the other. In reality, there are numerous routers and switches along such a path that introduce additional queuing and store-and-forward delay. They are there in part to provide cost-effective sharing of such a long distance connection. That is, other traffic can enter the logical link at each point there is such a router, sharing the bandwidth and cost of this link, cross country. However, each additional traffic stream introduces extra queuing delay and variance in delay, further departing from the memory bus ideal. Moreover, congestion can lead to packet drop and dramatically higher levels of delay, in fact unbounded levels of delay.

The cost of the transfer unit in addressing and switching pushes for a larger transfer unit than the memory bus, leading to variable-sized packets such as Ethernet 64 to 1500 bytes, producing additional delay and variance in delay. Also, the unit at the higher level, such as application level, is even larger, such as the whole file transfer used in HTTP.

Considering reliability, data is lost because of overload and failures. As an interconnection network scales, it becomes increasingly hard to avoid overload from a number of sources sending to the same place at the same time. Memory bus techniques such as preallocating the resource (i.e. request the bus first) introduce excessive delay because of the intrinsic delay, as above[4]. Even more challenging is the problem of multicast to a variable number of endpoints without congestion. Note: reliable broadcast/multicast is normally supported (and required) in a shared memory

[4]In this sense, it is a network that practices *admission control*, something which has been difficult to scale in large-scale networks.

multiprocessor over the bus interconnect. However, it is intrinsically non-scalable because the delay, jitter and cost increases significantly with scale.

Thus, as the interconnection network scales, expect more delay and expect greater variability in reliability. Note: higher bandwidth is not really as difficult to provide. For example, gigabit Ethernet to the desktop is basically available now. Also, *jitter* — variance in delay — can be an issue for some applications.

Scalable distributed system is a system that is designed to deal with:

- a network with varying delay, bandwidth and no reliable broadcast, with performance orders of magnitude worse than that of the memory system, at least on some dimensions.
- a variety of computer nodes, changing in number, configuration with no practically restricted bound on the number of nodes.
- Some nodes being up, some down, some links are up and down. Overall, it is never completely up or completely down, but a system-wide reboot is very bad.

Commonly, the network is *unreliable*, i.e. provides best-efforts delivery so packets, requests can be lost, unlike a backplane bus. E.g. Ethernet, IBM token ring. (Broadcast and multicast, if available, are typically even less reliable.) You can regard loss as part of variation in delay where a lost packet has infinite delay, especially recognizing that a packet delayed beyond some bound is considered lost, even if it shows up later.

Actually, "best efforts" is an overstatement. A network generally only makes a *reasonable* effort to deliver packets, reasonable relative to the cost of the protocols handling packet loss and the cost of improving the reliability of the network. I.e. a router may drop a packet because the output queue is "full", which arises because there is a bounded size to the queue, which arises because it is simpler and adequate most of the time to bound the size of transmit queues.

Categories of Interconnection Networks: Here's 3 silly categories of networks, whose names means little.

CPE Networks Historically, dialup access at 75 BAUD to 128 KiloBAUD, using RS232 Asynch, BISYNC, HDLC, ARPAnet, ISDN etc. and more recently DSL at 128Kbps to 1.5 Mbps. These support (slow) web access, file transfer, e-mail, etc. These have been replaced with DSL, cable, celluar, satellite, Wi-Fi and some prospect of WiMax. The overall trend is increasing (max) bandwidth, but they suffer from high and variable latency and reliability, some of which seems intrinsic in being exposed to the problems of wide-area distribution.

Distributed File System Networks 1-10000 Megabits, e.g., Ethernet is by far the dominant technology (and deserves to be) plus ATM at 155, 622 Megabits, IBM Token Ring, Fiber Channel FC-2 at 266 Mbits and upwards, and various antiques. 10/100 Mb is common, 1000Mb is here and 10000 Mb is coming.

— speed of network, basic packet size, reliability, scale designed around disk transfer and file sharing characteristics, with store-and-forward networking.

Distributed Memory Networks: 100 Mbits to 1-20 gigabits, Intel's NGIO network, HIPPI channels at up to 10 Gbps, Myrinet at 1 gigabit, going to 5 and 10 Gbps and more recently Infiniband — an industry consortium to specify a next-generation interconnection standard[5].

— specifically engineered (supposedly) for low delay, often using cut-through routing.

The first category is historical in name, but restricted limited communication is likely to remain with us. The third category is very expensive and niche because few applications care about really low latency, and there is enough performance coming from standard high speed data networks such as gigabit Ethernet that the market for specialized networks is being squeezed. You pay a high premium for low delay, and there is limited return in many applications because you still need to implement *latency-hiding techniques* even with the specialized networks. That said, database clusters such as Oracle RAC (with synchronous locking and transaction commit) and HPC clusters using MPI (with its barrier synchronization, etc.) significantly benefit from very low-latency interconnects, although it is not clear even these interconnects solve the performance issues in these cases, especially with scale.

As general trends, bandwidth will be continue to be limited[6] but delay (and the infinite delay of failure) will continue to be a big issue. Thus, latency-tolerating software techniques will become increasing important, if not critical. Once the latter are properly practiced, there is far less value in specialized networking technologies, such as cut-through. Commodity networking (e.g. Ethernet) will play a dominating role, even with HPC, here with 10 GE Ethernet.

1.3.3 Distributed System Software

The distributed system software is what allows or causes the set of interconnected computer nodes to function as a system. This section considers some of the desirable

[5] I believe this is largely doomed in the long term. They cannot do anything short of a whole network design to get the scale and flexibility they are after, and if they do, they end up with a specialized (hardware) network protocol architecture that is inferior to (and incompatible with) IP. In the short-term, Infiniband appears cheaper than 10 GE because of less buffering and different market dynamics.

[6] Some systems are trading CPU cycles for bandwidth by compressing data before transmission.

properties of this software.

The design of this software is a design-in-the-large problem. In fact, given that a distributed system can span thousands of nodes, perhaps it should be termed "design-in-the-VERY-large". Thus, the overall structuring/factoring of the system is critical, not just the selection of good protocols and algorithms to provide individual aspects of the functionality.

Choice of the infrastructure or framework API is an important issue. With the right API and functionality, the framework support can provide a significant amount of well-tested code, reducing the development time and cost associated with a distributed application or system development. Many applications are developed to a communication layer API, such as Unix/Linux sockets. I believe a key issue is providing better state-level support.

Distributed Application Design Steps

The shared state model of distributed applications and systems, depicted in Fig. 1.3 suggests there are three basic steps to designing distributed systems software, besides writing the application-specific logic.

1. Identify shared state and design the object interfaces to it, estimating or determining access behavior to this state and configuration of the state.

2. Implement shared state in one or more modules that handle consistent sharing of this state across nodes, using techniques matched to object interface semantics and communication facilities.

3. Distribute the objects according to processing requirements, state access, reliability and security issues.

Different applications and services have different shared state requirements and properties when distributed. The shared state maintenance can be tuned to these requirements and properties.

For example with a distributed game, each node may have objects corresponding to the local player plus some state associated with other players that should be visible. The simplest approach is to communicate all the player state to every node, but that is not satisfactory because player nodes may try to cheat. Thus, the state at a player node may have to be dynamically reconfigured over time based on what the player should have access to. This can also arise if the overhead of updating all the state on all nodes all the time is excessive, as arises with large-scale games.

Modularity

Being a large and complex design challenge, it is important in a distributed application/system to provide a modular structure to the design in which each module is well-defined and encapsulated and implementable, and interactions between these modules introduce limited coupling yet allow appropriate functionality, performance, reliability and security (standard motherhood).

There are three basic issues:

1. What functionalities, state and properties are sensible to encapsulate in a module and what functionalities need to be provided as part of multiple modules (possibly as a common library).

2. In what module should the state and functionality be placed (assuming it passes the above test)?

3. How should that module interact or coupling with others?

As an example of the first issue, should reliability be an aspect of every module or provided by a separate *reliability* server? This example seems like a clear case in which a separate module is not feasible and the property requires functionality in several modules. However, what about naming? Should there be a name service module or is naming aspect of every service?

Considering the second issue, the end-to-end argument (Saltzer84) suggests making functionality part of application/service-specific objects unless it is an optimization that fits at some lower-level. So, the question in part is how much the lower-level can do for us as an optimization.

The third issue concerns which modules call this module and which modules it calls.

These issues are interrelated. For example, if an element, by functional definition, requires very tight and expensive interactions with other elements in the module, the functionality probably does not make sense to put in a separate module. For example, a process migration server would likely have to modify the internal state of many modules that are supporting process execution, and thus have state associated with the process. The knowledge that this server would require of these other modules and the resulting tight interactions appear to make such a separate server infeasible.

However, this is the same problem of designing for modularity that occurs in programming in general. Distribution just enforces an extreme form of enforced modularity. The assumptions about, and interactions with, other modules are limited by communication properties of high/unbounded delay, bandwidth restrictions and independent failures and recoveries that can occur between nodes.

To address these issues, Chapter 3 explores the object-oriented structuring of distributed systems. We take the notions of object, interface and multi-object structuring techniques developed in object-oriented programming in general and apply (and extend) them to distributed systems. A key part of object-oriented structuring is defining interfaces that define the interactions between objects. In the distributed systems context, this emphasis leads naturally into a *protocol view* of systems, in which what goes across the wire defines the system, not the specific software.

Another key strategy for distributed systems modularity is insulating application software from the network by providing network transparency in the framework abstraction.

Network Transparency

The distributed system software ideally provides an execution environment that is *network transparent*. The *execution environment* consists of the names, operations and data that a program uses (directly) during execution.

Being *network transparency* means the execution environment provides an abstraction that is the "same" whether the objects an application is accessing are available locally, remotely or distributed across several nodes, both locally and remotely. — the network is not visible. This includes the system providing global names for objects, i.e. ones that mean the same regardless of location.

Example: object-oriented remote procedure call (OO RPC) allows calls to objects that are remote or local in the same way, with no change in source code for the two cases. (In fact, the code can do both.)

Example: with Unix/NFS with cluster of homogeneous of workstations.

Use a file naming convention to get a common global view of file names.

— e.g., "/-" as 'superroot', with all file systems mounted under on all machines.

— coordinate user numbers, groups, etc. across all machines.

Problems:

1) /tmp — is still per-node, and by statistical access, *should be* per node. There are also per-node config files, including those in /etc, /bin and paging/swapping storage.

2) process identifiers are local to each node, file descriptors local to each process. That is, there is no *location transparency* .

3) Consistency of data. In general, it is relatively straight-forward to implement a mostly network transparent system. It is the last 5 percent of network transparency that is hard! For example, some network transparent device access is difficult to implement. E.g., frame buffer. Solution (?) in V, Amoeba, etc.: Most programs

access devices via resident servers, e.g., display server for frame buffer such as the X server.

However, network transparency introduces an engineering trade-off. The 95 percent network transparency provides most of the benefit for a small amount of the cost required to get to 100 percent, so is generally sufficient. Differences in hardware also impact network transparency. E.g clock granularity, display capabilities, processor speed (and even instruction set). And, many exceptions to network transparency need to be provided for administration/management. Performance transparency is basically impossible in general.

Solution (?): make assumptions about/place restrictions on transparency. This is similar to the approach to conventional program portability. I.e. you restrict what you do to a portable subset. Also, you should recognize that most of the benefit is achieved when it is generally not necessary for applications to know about the network or remote operation. It is harder and less valuable, and perhaps not even feasible to prevent some module from determine whether something is local or remote. Simply stated, this means that application logic should not be forced to deal with the complications of remoteness and network operation, but is not precluded from determining aspects of distributed operation. I.e. one can avoid (the need to) pollute a lot of the application logic/code with distributed systems considerations and details.

Fault Tolerance

The reality is that nodes and network connectivity can fail in a distributed system. The challenge is making the application/system tolerate these faults and continue to work. More nodes, more links and thus the larger the network and more coupled activities leads to increased probability of individual node/link failure. The challenge is structuring the software to be modular and network transparent and still tolerate faults.

Redundancy of processing, storage, communication (theoretically) decreases probability of total system failure. Redundancy also increases the required consistency/synchronization. The software should provide the ability to survive individual failures, especially with scale, at a reasonable cost. There is trade-off between performance and consistency. And, as we observed earlier, only the "ends" know enough of the semantics to tolerate inconsistencies.

As an extreme reliability example, there is on-board processing in the space shuttle, rather than making it a dumb X-terminal equivalent, so that the control (and astronauts) are not necessarily lost if the communication is.

There is a key tradeoff between availability and consistency. E.g. Can I access a cached copy of file immediately or have to wait for file server to come up to

ensure I am getting the latest copy, i.e. consistency. Allowing continued operation with partitioned system increases availability at a cost in consistency, or reliability[7]. Most of the real world now favors availability over consistency, yet most research has focused on strictly consistent approaches.

Scalability

Distributed system software should provide good performance, at least relative to that possible with the provided hardware base. A particular concern is how it performs with scale. I.e. how do message/storage/processing costs grow with distance/delay, number of participants, and heterogeneity of system, both for applications and services? The design should also avoid bottleneck resources. E.g. single "global" name server.

Communication support and constraints can have a major effect on design decisions. For example, if cross-country interaction is needed, then a 30 milliseconds roundtrip (or more) is fundamental, requiring extra software complexity for *latency tolerance* techniques.

Low data rates → find minimum *cut points* in data flow in system.

E.g. local terminal display with remote compute vs. local computing with remote file service.

Large delay → minimize synchronous interactions.

E.g. streaming file transfer, versus conventional request-response RPC-based I/O. New issues arise with high-performance compute clusters, with large amount of data movement, in particular congestion.

A relatively new direction is input data-intensive applications, vs. internal data intensive applications. One example of the former is Google's MapReduce framework where large amounts of input data is processed compared to a relatively small amount of what you might call *historical data* residing inside the system. Another example of an input data-intensive application is an autonomous vehicle, where sensors such as cameras, laser range finders, radar, etc. can provide a huge amount of data input. The system needs to be optimized for handling this huge flow efficiently. In contrast, the conventional system is one centered on a central historical data storage such as a file system or database system and is optimized for data flowing to and from this central repository. Of course, an input data-intensive application such as one that first stores the input data on disk can blur the distinction somewhat, especially if it does some preprocessing. In any case, the true input data-intensive applications tend to be processing one or more *streams* of data, rather than requesting specific records from a central data repository. Thus, another distinction is

[7]Some authors using reliability to mean correct operation, which obviously corresponds with consistency.

based on the unit of request. An input data-intensive application has modules that request a large, if not unbounded, data set, often reading it from their initialization to termination, if the latter ever occurs. In contrast, the conventional application requests input in units of records or relatively small files that may be opened for a relatively short period of time. As a result, the conventional is relatively random access over time where as the data-intensive tends to be more stream-structured.

Administration, Management and Politics

View: the operating system management for "mutually suspicious users" problem from the 60's is far from solved. This management problem (and reduced cost of hardware) killed Grosch's "Law" and the fantasy of the *computing utility*. L. Lamport: "Distributed computing is having a machine you never heard of bring your work to a halt."

Management: (1) know what's going on in your system, (2) determine the cause of misbehavior when detected in above, (3) fix/reconfigure the system to correct for misbehavior. Thus, you need to be able to monitor and control your distributed application or system, including monitoring and controling the consumption of resources. E.g. can my network be flooded with transit traffic between nets A and B if I connect to both net A and net B?

One approach arises with the so-called grid approach, in which cluster management provides for monitoring and control of the cluster resources. Vmwares V-Center is one example that plays a role in virtualization in data center.

Another aspects is trying to provide loose coupling and strong separations. ("Good fences make good neighbors" Robert Frost.) Loose forms of coupling and autonomy helps but limits the functionality and performance. Examples include: remote terminal access, electronic mail, FTP. However, with loose or no control over the system, it is difficult to ensure any particular properties. Generally, ownership is intrinsically tied to control, and distributed ownership means no central control unless the owners cede to some degree to an umbrella organization.

In this vein, the Internet an interesting example to consider as a confederation for ISPs as independent/autonomous entities. In this environment, ISPs negotiate peering arrangements and agreements, driven individual ISP requirements and needs as an independent enterprise. This leads to decisions and interactions that are driven by business or political considerations more than technical. In particular, a service is provided by one ISP to another, driven by the Adam Smith "invisible hand" of commerce. In this vein, you can view the economy as a large dynamic distributed system, with responses to requests driven in part by business factors. In this view, the interactions are often multi-round negotiations and far less efficient than the simple direct client-server interaction. In fact, in this context, you might regard

a client-server interaction as better characterized as client-slave. I.e. the server is basically a slave to the clients, doing what they request because it is so programmed, not because of some innate desire or other compelling force.

1.4 Example Systems

An application is rarely, if ever, intrinsically distributed. Distribution is just the lesser of the many evils, or perhaps better put, a sensible engineering decision given the trade-offs involved. For example, you might argue that terminals, such as point-of-sale terminals, are intrinsically distributed because of the need to have sales transaction support at each store. However, terminals can be extremely simple or "dumb" so the degree of distributed processing can be really trivial, basically just display and keyboard input[8]. In fact, point-of-sale terminals are currently mostly very dumb terminals that call into a central transaction processing site.

AOL, as another example (at least in its original form), was a good illustration of a distributed service that is implemented in a higher centralized way. Basically, they "back-haul" all the traffic to a large data center in Dulles, because it has been cheaper to pay for the bandwidth than to distribute the people that manage/operate the service. (There is also a disaster recovery fail-over site.) In general, it seems attractive to put "all your eggs in a small number of baskets and take very good care of these baskets", i.e. centralized as much as makes engineering sense.

The following examples illustrate the diversity of distributed applications and systems, their requirements and general structures. In these examples, think: What's the shared state, what are the coupling/consistency requirements, and what are the protocols for sharing state and achieving this consistency.

1.4.1 Software Development System

Some portion of the research effect in the 1980s, inspired by the introduction of Local-Area Networks (LANs) was actually focused/motiviated by distributed software development, e.g. separate PCs or workstations providing interactive editing and compiling accessing shared state stored in a file server.

Shared state: source code and related files, such as documentation, make files, etc. and the tool chain in general. That is, the primary shared state is in files, so the key functionality required is a distributed file system. Thus, files are the primary objects of interface and file access protocols and file consistency protocols are the key protocols.

[8]The early SAIL timesharing system developed in the 1960's at Stanford University used a shared video display controller so that terminals were really simple — even their displays were driven from a video link from the central machine.

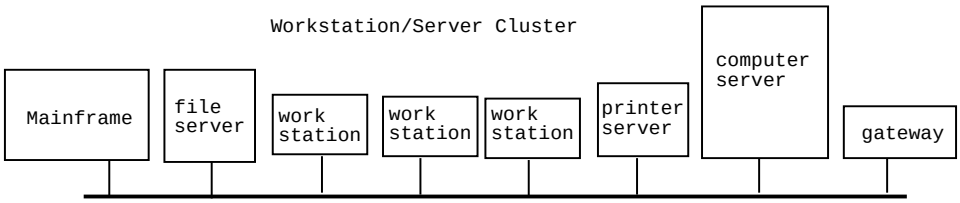

Figure 1.4: Software Development System

Requirements:

- Interactive editing, browsing, debugging, graphics, etc.
- Safe, recoverable storage of files, databases.
- Moderate processing for compilation.
- Possibly, big crunch for simulations, verifications, testing.

PC/Workstations — for editing, compiling, debugging, etc.

File servers for safe storage — keepers of the shared state.

Dedicated parallel machines for simulations or else the PC/workstation cluster itself for distributed parallel execution, *assuming* the necessary OS and simulation support, and a sufficiently fast network.

Protocols: IP/TCP, NFS, lockd, DNS, AFS, etc.

1.4.2 Distributed Banking System

Shared state: customer accounts, bank general ledger, logs of activity, etc.

Requirements:

- Interactive ATM, teller support, loan officers, etc.
- Transaction processing, report generation.
- Database management, including logging and journaling.

Options:

1. **Totally decentralized** — PC/workstations only, where the shared state is distributed across teller machines. Fig. 1.5 depicts this configuration.

The key problem with this configuration is that every PC has to be protected both for security and for reliability because of the state it holds. For instance, it should not be possible for a teller to reboot the machine in some unprotected "superuser" mode and have arbitrary access to data, even just the data stored locally. Similarly, the machine should be protected against theft and accidents such as coffee being spilt on it.

2. **Centralized** — mainframe and terminals with all the shared state stored on the mainframe.

Fig. 1.6 depicts the structure of a traditional mainframe-based banking system.

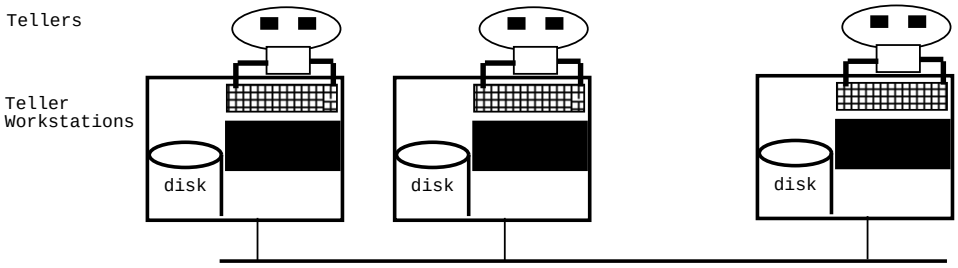

Figure 1.5: PC/Workstation-based Banking System

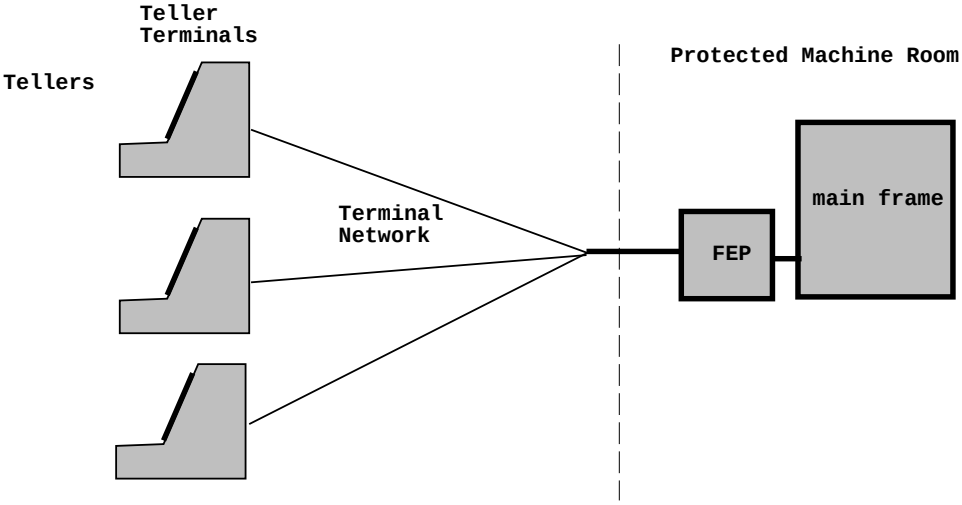

Figure 1.6: Mainframe-based Banking System

Mainframe as performance, reliability bottleneck, large "forklift" upgrade unit (all or not at all).

— communication and interactive overhead to mainframe.

— most airline reservation systems follow this model.

Mainframe has to satisfy storage performance requirements as well as user response requirements, running interactive and batch applications.

3. **Workstation/Server Configuration**

Fig. 1.7 depicts the structure of a combined workstation/server banking system.

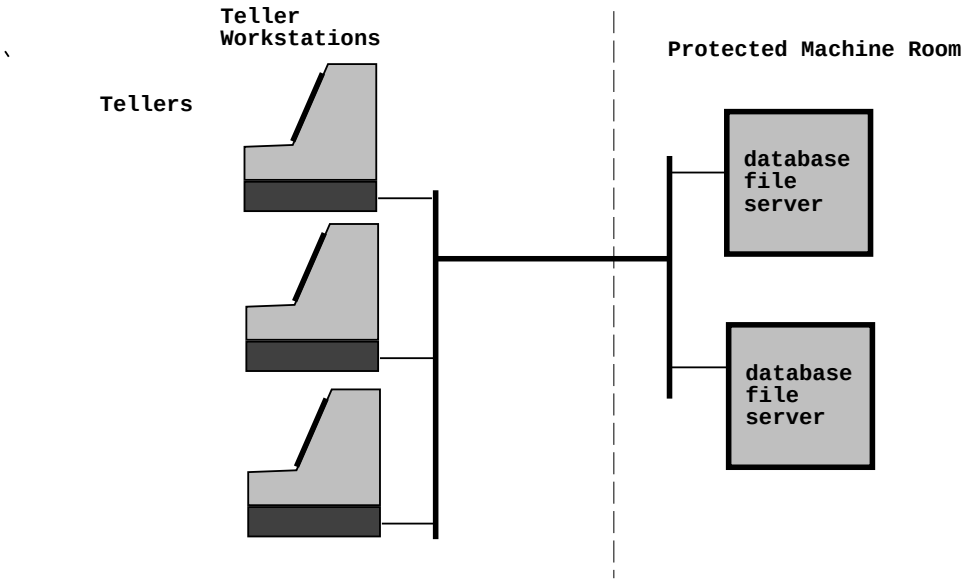

Figure 1.7: Workstation/Server-based Banking System

PC/Workstations — handle interactive tasks plus some portion of transaction, database and report generation processing.

Another modern possibility is a SUN Microsystems Sunray — a thin-client display and input without local storage or management problems. Administration of machines is now a dominating cost. In this, you can scale by replicating the servers, but having them share disks (JBOD) over a SAN (storage-area network).

File/database servers — reliable, secure storage plus some degree of processing.

Issue: partitioning of processing between workstations and servers.

1.4.3 Embedded Real-time Distributed Systems

Appliances and vehicles, including autonomous cars (e.g. the DARPA Grand Challenge winner Stanley) and unmanned aerial vehicles (UAVs), are using a distributed implementation internally to provide fault-isolation between components and, in some cases, adequate processing and response time. As an ilustration of the former, flight critical software is required to meet DO-178B certification. Two separate functional processes handling functions at different safety levels cannot run on the same physical node unless there is an operating system that guarantees adequate protection and isolation between these processes. Consequently, many systems are distributed with each function is implemented on a dedicated node. (However, because these components are connected by a network, they are exposed to potential failures induced by bad data from other nodes.)

As an illustration of an autonomous vehicle system, a vehicle may have a large number of sensors that are outputing raw data that needs to be processed and calibrated in real-time, requring both high throughput as well as low-latency for the stability of the filtering algorithms. Furthermore, a control system for the vehicle needs to respond to changes in inputs to maintain the trajectory and stability of the vehicle. Finally, in the case of an autonomous vehicle, a planning module needs to responsively recalculate a path to follow based on changes in the environment, such as sudden perception of an obstacle. Thus, both performance and fault-tolerance dictate implementing these systems as distributed systems, and thus addressing the challenges we explore in this course.

In this domain, there is a large amount of state corresponding to sensor input data and derivative information such as position, velocity, etc. that needs to be shared across the modules. Ther is also control and configuration that needs to be distributed from planning and control modules to the actuators.

With multi-core making it feasible, if not inevitable, to have a distributed system on a chip, and embedded systems becoming more complex, whether a smart phone or an autonomous vehicle, distributed systems technology is expected to play a strong role in future embedded systems.

1.4.4 HPC Cluster

A cluster of commodity blades can be used as a distributed parallel computer system. In fact, from the economy of scale from using mass-produced industry-standard servers, such a cluster may provide computing cycles more cost-effectively than a specialized multiprocessor or massively parallel computer. These mass-produced units may also be more reliable and easier to upgrade than the previous generation so-called supercomputers.

As a recent development, various companies are now marketing *blade servers*, high-density boxes that contain multiple complete computer nodes — processor, memory, network and possibly disk — packaged to save on rack-space, network connections and management compared to separate 1RU servers. In using these cluster-based systems as a parallel machine, the shared state is the state of the individual parallel programs being run on such a cluster, as well as the operating system state.

There has been a dramatic growth in: 1) the use of clusters compared to shared memory multiprocessors, 2) use of x86 processors vs. other architectures and 3) use of Linux per node versus other operating systems, including specialized operating systems. A popular approach is to use MPI (Message Passing Interface) on each node as the API used by the application. Google's MapReduce is another approach that is indicative of directions in this area, although that is more focused on data mining. (Hadoop is an open-source version of basically the same cluster computing application framework.) The potential for computing power in the area is enormous. Faster, muti-core CPUs are coming, memory is decreasing in case, and there is the prospective of lower cost interconnect using Infiniband and 10 (or more) Gbps Ethernet.

More historically, Treadmarks (Amza et al. 96) is a software system that provides an abstraction of shared memory across a cluster of hosts on a network, allowing for execution of parallel programs in this environment. This shared state uses distributed shared memory protocols that lead to tight coupling among the participating nodes. This system is discussed further in Chapter 2.

The V Distributed System (Cheriton88) is an example of a system providing some support in this direction. It is a distributed operating system providing transparent transactional communication between machines connected by local network (as well as intra-machine). It was developed at Stanford (1981-88). V supports process migration and distributed scheduling. It was used for distributed parallel computation. However the distributed scheduling did not provide specific support for multiple parallel applications. The state sharing was handled in terms of messages with no uniform handling of state beyond system-wide protocols for naming, and I/O.

Also, there has been some success with software facilities such as *PVM* (*Parallel Virtual Machine*) from Oakridge, more recently the X10 language developed by IBM research and DryadLINQ from Microsoft Research.

It provides a platform-independent computing environment, and has been used to write parallel scientific applications that have run on clusters of workstations as well as on distributed memory parallel machines such as the Intel Hypercube and Paragon. Chapter 11 discusses these issues further.

After extensive research work in the 1980's in workstation clusters, there was a bit of a lull in activity as individual hosts became far more powerful, decreasing the

need in general for parallel systems. Also, during this time, network speeds did not increase, making the network more and more a bottleneck to performance. Moreover, research distributed systems failed to displace conventional operating systems, making distributed parallel computation limited and in some cases, inefficient.

The various peer-to-peer computing efforts, such as seti@home (see: http://setiathome.ssl.berkeley.edu/) can be regarded as similar uses of distributed systems. However, in this case, the data rate to each machine is generally very low, restricting appropriate applications to highly partitioned tasks with very low communication requirements. Other peer-to-peer efforts such as gnutella are focused on distributed storage, not computation and seem to be primarily motivated by the desire to provide unauthorized access to copyright material without a central target for the copyright holders to attack. I have not identified any compelling reason for a so-called peer-to-peer system as an economic enterprise except to engage in illegal activity.

1.4.5 Server Cluster

Large-scale Internet Server clusters have revived interest in cluster computing because of the need to scale web sites, including search engines, to serve large numbers of users. Moreover, in 1994/95, conventional cost-effective LAN technology (Ethernet) really started to change in performance, to 100 Mbps, and then to 1 Gbps, with 10 Gbps now being deployed and higher speeds in the future. As a particular instance, Inktomi came into existence as a search engine company building a scalable, robust search capability on a cluster of machines, using commodity hardware and highly parallel search to get cost-effective performance.

The economic importance of search motivated the significant investment to tackle the complexity of distributed parallel processing with techniques highly specialized to search. See the reading by Fox et al. for more details on the cluster technology.

In essence, a cluster appears on the Internet with a single address corresponding to a *virtual host*. A front end switch or router (such as Cisco's LocalDirector, Foundry's ServerIron and other products) or an actual host machine responds to that address, and dispatches each incoming connection to one of several front-end servers machines. These machines handle the request from a shared database, and respond back to the requesting client. In some cases, the handling of a client request can be partitioned further. For instance, with the Inktomi server, each request is effectively multicast out to another level of search servers that each perform a lookup on the search query in parallel, returning the results to the server handling the request for merger of the results.

Overall, there is a widely used three tier model, namely:

- **Tier 1:** web servers (or XML), to handle incoming customer requests, sometimes called the presentation layer.
- **Tier 2:** session and business/service logic servers, including index servers in the case of search engines.
- **Tier 3:** database management systems, performing the actual e-commerce transactions, containing the most sensitive data and often the most scalability-challenged portion.

Looking at these systems from a shared state standpoint, tier 3 actually persistently stores the critical data, whether the e-commerce transactions, customer records or master copies of the content. Tier 2 provides session state which is not as critical but needs higher performance access, as well as caching state from layer 3, e.g. memcached. Tier 1 caches quasi-immutable state as well as provide dynamically generated state from that provided at Tier 2 and 3. e.g. a Facebook page built from memcached key/value pairs.

1.4.6 Strategic Defense System

Consdering a very different type of distributed system, one other topic in the 1980s was the development of the so-called strategic defense system, the ultimate distributed system. Here, the goal was to detect, track and destroy 'lethal objects' using space-borne and ground-based sensors, weapons and computers, e.g. counter a potential Soviet nuclear-tipped inter-continental missile attack by a missile defense system.

Distributed because:

- sensor platforms — in low and high (geosynchronous) orbit.
- weapons — ground-based if possible but some in space if necessary.
- battle management and C^3 (command, control and communication) — ground-based President, some control in sensors and remote weapons.
- components distributed for survivability.

Shared state: sensor data, position of sensors, position of weapons, deployment and deployability of weapons.

Approaches:

1. Centralized — N supercomputers connected by Hyperchannels,

2. Davy Crockett — totally decentralized. Every sensor/weapon platform for itself.

3. Coordinated/cooperating but with some autonomy.

Problems with each (by number):

1. Not survivable — loss of communication or loss of the supercomputer site itself.

2. Double targeting — subject to weapon exhaustion. Also, target discrimination — decoys in mid-course.

3. Distributed, cooperative, provably reliable, secure algorithms and protocols are needed!

Here, the shared state is the sensor data and tracking of threats plus the state, position, capability, etc. of all "asserts" plus command and status information for all these components.

1.4.7 Distributed Interactive Simulation (DIS)

Distributed interactive simulation (DIS) creates a real-time virtual world in which participants can interact with each other, the simulated environment and simulated entities. A significant effort has been underway in the military to develop DIS technology for training and weapon system evaluation in small, medium and large-scale exercises. Some of the entities in a military DIS are depicted in Fig. 1.8. Each partic-

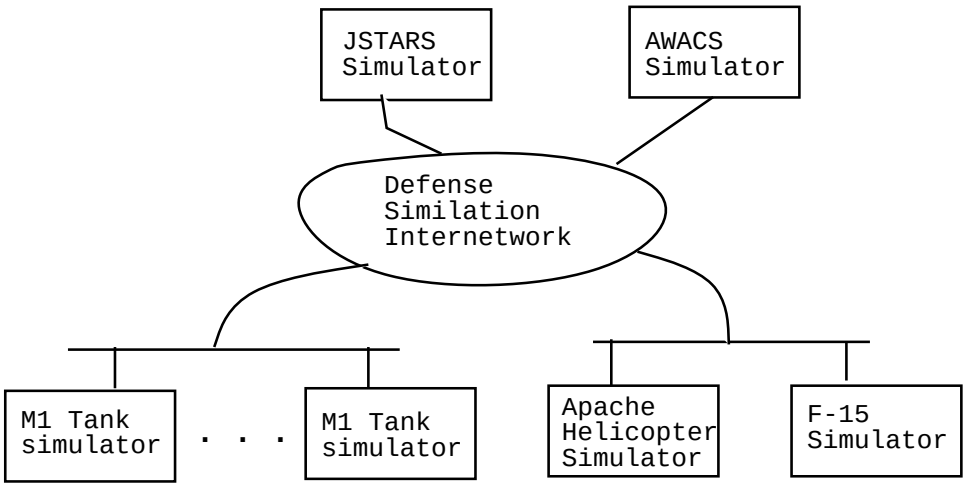

Figure 1.8: Wargaming: Distributed Interactive Simulation

ipant in a vehicle or post with computer-simulated visuals, audio, instrumentation, IEW (intelligent electronic warfare), etc.

E.g. M1 tank with OTW (out-the-window) visual simulation of visible environment.

— Computers connected by a network exchanging information to communicate and synchronize on position, orientation of vehicles, firings, damage assessment, etc.

Shared state: Current position, orientation, status of each entity in the simulation. Also, terrain data, atmospheric conditions, command and control information, even radar data, etc.

Distributed because:

- Participants are distributed, and I/O between participants and simulation is very high. E.g. 20 frames per sec of high-res. 3D graphics from simulation state to participant screen, possibly multiple screens.
- Separate manufacturer for different detailed vehicle simulators, specialized I/O, and proprietary techniques, so separate "boxes".
- Scalability — multiple computers for increased processing capacity and reliability with scale.

Coupling: loose in failure, but coupled for real-time update and response.

Protocols: Current DIS (standard) approach, based on PDUs (protocol data units):

- Each simulated entity broadcasts changes in its position/orientation periodically, including velocity and acceleration, so-called "appearance PDUs" (1 second interval is typical).

  Usually one entity per network node, e.g. F-15.
- Each entity sends out a PDU for extraordinary events, e.g. fired missile, damaged, etc.
- Each entity dead reckons position of all other entities (smoothly extrapolating positions), updates state based on received PDUs, and performs local hit/collision detection and damage assessment.

Dead reckoning gives smooth animation with minimal network load.

The network is the only "single point" of failure.

Any simulator can participate provided it "speaks" the DIS protocol.

Largest exercise to date: almost 10,000 participants. The challenge is to scale much larger in many dimensions, including number of participants as well as granularity, flexibility, and so on.

Extensions:

- *Scalability:* broadcast does not scale, multicast filtering based on location, "intelligent gateways", etc.
- *Dynamic terrain*, e.g. Is a bridge an "entity" or how does it blow up? What about holes in ground, effects of rain, etc.
- C^3I : command, communication and control infrastructure within the simulated environment.

Also, support for SAFOR/IFOR (semi-automatic/intelligent forces) as opponents as well as subordinates.

Overall, DIS and multi-player games developed somewhat simultaneously but with multi-player games, centralized control has become important to prevent cheating, as well as to minimize communication costs and allow upgradeability, as compared to academic notions of direct communication between game nodes. However, a single central server is a limit on scalability. One can scale by introducing partitioning of players/participants and partitioning by state (such as some servers provided imaging, video, graphics, etc.). One can also have tiers as with web sites. For example, normal animation can be generated by an tier 1 server but a purchase of virtual clothing, land, etc. would go through to the tier 3 server. The challenge of scaling multi-player games remains a significant engineering and research challenge.

1.4.8 World-Wide Web

The world-wide web is phenomenon that caught a lot of distributed systems people by surprise because it is so trivial in many ways.

The name space of *Universal Resource Locator URL*'s is formed just from a protocol name, DNS host name and Unix-like file name within the host. This host-specific/based naming of objects was disparaged by distributed systems researchers, but seems hard to escape from. It was clearly built from what is there. (However, using DNS tricks plus *virtual hosts* implemented using products such as Cisco's Local Director, a single DNS name can map to multiple server machines.) As a side note: URLs embed the server name in the URL, the simplest approach. A more sophisticated approach is having a mapping from name prefix to server name, and locating the object in by this three step process, i.e. name prefix to server name to server, to which you then send the full object name. More on this in Chapter 5.

Web servers implement a *HyperText Transfer Protocol HTTP* for accessing remote information over TCP connections (or similar). It is really very simple, primarily consisting of `GET`'ing a file identified by a URL and `POST`'ing forms plus (less used) commands for linking, unlinking and deleting of data. There is almost nothing "hypertext" about the protocol — hyperlinks are handled by the browser. In many

ways, it is a simplified file transfer protocol, i.e. `GET` this HTTP header-specified content file, as compared to ISO's FTAM.

The presentation protocol is basically the *HyperText Markup Language HTML* layered on ASCII. HTML is also a rather simple formatting language.

Client browser programs such as the Google's Chrome, Firefox, and Internet Explorer, derived from the original Mosaic program, allow one to follow links transparently to data across machine boundaries, but that is basically a local browser function. (There is no provision against dangling links or pointers except for careful human management.) It provides a cosmetically appealing front-end to a relatively simple server protocol and mark-up language. Java effectively takes the presentation capabilities at the browser beyond simple document display and provide animation and more complex interaction processing.

The power of the Web comes from putting these "simple" components together, building on the connectivity provided by the Internet. They form a system with remarkable power, addressing the challenge of "publishing" information. The search engine capability further fills in the picture by providing a powerful means of locating information that is Web published. For instance, with Google as your interface to the web, the shared state appears to be a significant portion of human knowledge and information, ranked by a large amount of human knowledge as well.

The web is also a good example of how difficult it is to identify the combinations of technology that produce a compellingly useful system. Also, it illustrates that this sort of "hit record" is often from novel combinations of relative simple existing components, not a complex new mechanism. It's "the power of the hack" — a simple insight will kill you if your competitor comes to it first.

1.4.9 Service-oriented Architectures

Building to some degree on the success of the web as information server, there is a trend in industry towards the so-called *service-oriented architecture(*SOA). The key idea is to recognize a facility as a *service*, meaning that it can be accessed by a protocol from potentially multiple applications, some of which are unanticipated/unknown. For example, rather than building a navigation component into an application so it is only available to that one application, it can be provided as a service so multiple application can use it. You might ask: why not just make this a library (function) and allow any application to link in this library? In some cases, the component is too large to link in, does not run on the application node for other reasons, is not allowed to for license reasons, or requires excessive startup time to access the required data compared to providing as a service. Moreover, it may not be feasible to provide the required fault-tolerance by linking it as a library, versus multiple redundant servers. Also, sometimes there is a security (restricted data access) issue with providing as a library. For example, the navigation capability may

require access to a database of geographical information in a relatively fundamental form that cannot be made available in general to applications. And, of course, in some cases, the functionality would work fine just as a conventional library. The web trend of providing services that are used as part of a *mashup* is similar.

The SOA direction is being promoted in conjunction with an enterprise services bus (ESB), a common mechanism for naming and accessing these services. This reduces the amount of mechanism required by applications to access the different services. It also means there is a new (proprietary in some cases) platform to lock customers into. Besides the web services and SOAP support, most of the offerings have basic message-level mechanisms. Relative to our earlier classification of systems into composition and decomposition, one might regard the SOA as bridging between these. An application is developed, in part, as a composition of various service nodes yet the service nodes were designed to be used by various distributed applications. Alternatively, adopting an SOA is basically adopting a decomposition of the overall enterprise information systems and applications using an ESB, and pushing the various enterprise services into being provided in the service mode of the ESB.

1.4.10 Unanticipated Distributed Applications

In many cases, you start with a required application and develop protocols to support it. However, in many other cases, distributed applications have arisen that make use of existing protocols, often in ways that were not anticipated by their designers. To cite an old example, Alex (Cate and Gross92) was developed as an NFS file system implementation that provides Unix filesystem-like access to anonymous FTP-able files. File pathnames are the (reversed) FTP host name followed by the path name of the file, relative to the anonymous FTP directory at this host. For example, a file on host `ftp.cse.ucsc.edu` in directory `/ftp/pub/tr` has an Alex name of `/alex/edu/ucsc/cse/ftp/pub/tr`. Alex automatically retrieves and caches files and directory structures on demand, and flushes items from a local disk cache after some period of non-use or running into disk limitations.

The `netfind` program, discussed in Chapter 5 is another example of this type of wide-area distributed application. This program is basically a directory service that supports locating people and their e-mail addresses on the Internet.

The World Wide Web, Alex and netfind are all built on top of existing Internet protocols and and services, providing a new service not previously envisioned during the design of the Internet. This type of development contrasts with the complete and integrated design that researchers often develop, (and discuss in courses like this). Unfortunately or perhaps fortunately, large-scale systems evolve somewhat chaotically and with lack of central control and planning. (The ISO OSI experience demonstates that some amount of anarchy is good compared to a large amount of committee.)

There are articles on the "education of a hacker", focused on reverse engineering software, learning how to probe, test and compromise, discovering how the software works or does not work, rather than the theory of how it is supposed to work, as is taught in formal education. Arguably, the hacker education model is better at inspiring "creative" use of existing software systems versus the conventional approach.

1.5 Overview of Topics

The rest of the book develops the key topics of distributed systems, starting with the next chapter dealing with distributed shared memory. (Not all of these topics will be covered in this offering of the course.)

Distributed Shared Memory

A distributed shared memory provides the abstraction of shared memory across two or more nodes in a distributed system. It is a familiar model for maintaining shared state local to one node, and is thus attractive to provide in a distributed environment. This chapter discusses the protocols and techniques for implementing DSM and associated issues such as performance and reliability. It also introduces the notion of specialized consistency protocols, leading up to a general theory of state sharing, ranging from direct communication to conventional shared memory. Here, the shared memory can be represented as an object, with the operations of read and write.

Object-oriented Distributed Systems

Chapter 3 describes the structuring of distributed systems as large-scale object-oriented systems, in particular the structuring of the shared state and the interfaces to that shared state in terms of objects.

The shared state of a distributed systems is encapsulated as *objects* that are accessed according to one or more *interfaces*. Object-oriented procedure call is used to access these objects through these interfaces. The protocols of a distributed system then are basically generated from the object interfaces and the protocols to be used. The protocol view of a system is basically the vertical cut of the traffic over the network. We also explore some interface design issues, including the recursive structuring of interfaces to provide uniformity of interfaces. This contrasts with the strictly layered approach.

The chapter also briefly considers some of the alternatives and conventional approaches to implementing distributed objects and providing infrastructure support, such as RPC, messages, distributed file system and distributed shared memory.

Clocks: Real-time and Logical

A clock is an object that maintains a linear monotonically increasing measure of time, real or logical. This chapter considers the techniques for implementing clocks and the uses of clocks in distributed systems. Real-time clocks are examples of shared state that can be maintained *sufficiently consistent* using approximate agreement mechanisms. Thus, clocks are also an example of problem-specific techniques for maintaining shared state in a scalable way, These are also the quintessential example of real-time processing in distributed systems.

Directories

A directory is an object that provides mapping for a name or identifier to the object, or an object reference. It also provides query and management operations for names, such as listing names in a given directory. This chapter considers both internal naming and external naming. For the latter, it addresses how to structure (global) name space(s), interfaces to naming facility, and protocols for implementing naming, local, regional and global. Implementation issues related to security, reliability, etc. are also considered.

Transactions

A transaction is an object that effectively *objectifies* a group of actions as a unit such that the consistency is enforced over that set of actions, rather than on each individual action. We regard the transaction is a batching mechanism for consistency, spreading the cost of consistency over the group of actions, rather than on each update, as with distributed shared memory. It also provides support for more sophisticated consistency, such as the abort capability, and the recovery support.

The chapter briefly discusses knowledge-based reasoning about distributed systems.

The chapter also briefly considers various approximate agreement approaches that are based on batching, or looking across a collection of actions for sufficient consistency.

Accounts and Authentication

An account is an object that designates the principal and the resource consumption funding associated with operations. This chapter focuses primarily on the hard and known aspect of accounting, namely authentication. Authentication also provides the basis for protection, which can also be viewed as an aspect of accounting or at least controlling access to resources.

Protocols and Presentation

This chapter and the next consider the details of supporting serialization and deserialization as well as interacting with a transport layer. This chapter considers the design of presentation protocols.

Distributed File Systems

A file server is an object that provides operations for creating, deleting and managing file objects. This chapter discusses issues in the design of the interface to a file service, particularly to facilitate extension to general I/O, the characteristics of file and the impact for file caching and file consistency protocols. It also considers various example file services.

Internet Content Distribution Systems

This chapter looks at techniques being used to distribute content over the Internet, layered on top of TCP/IP, viewing web proxies, content routing, etc. as a problem-specific techniques optimized for the needs of content distribution.

Schedulers

A *scheduler* is an object that supports the creation, deletion and management of processes, including their address spaces. This chapter covers scheduler structuring and implementation, scheduling policies and issues of fault-tolerance and recovery.

Semi-Autonomous Distributed Systems

This is a new chapter, considering systems with higher degrees of autonomy, such as a fleet of (semi-)autonomous vehicles. This also includes the operation of the Internet, which is basically a federated collection of routing domains.

Future Directions and Issues

This chapter discusses potential directions and issues for the future. It is all speculation. However, it is interesting to note that from the earliest days, file transfer has been the driving force between networking. In the 1960's, computer networking was motivated largely by the desire to move files between machines, avoiding tape drive incompatibilities. Simiarly, email (which is basically file transfer) was one of the big successes of the ARPAnet. In the 1970s and 80s', the focus was largely on

accessing files on a shared file server. Then, in the 1990's, the web emerged as a big revolution, but stripping away the cosmetics, it is just file access was dressed up by browsers, with HTTP as a simple file transfer protocol. At present, IPTV and video over IP are hot topics, yet video is just more file-like downloading/transfer.

1.6 Concluding Remarks

Managing shared state is the key challenge for distributed systems, in particular how to maintain sufficient consistency and trustworthiness to meet application requirements while not incurring the cost and dependence on communication that "excessive consistency" can lead to. As with many new areas, there is a tremendous amount of confusion and misguided efforts, in research, education and industry in this area, in particular, a misguided focus on communication layer to the detriment of focus on state and state consistency protocols[9].

Scalability is what makes things interesting: most things work at small scale. The challenge is providing for a success — ensuring the system can scale (cost-effectively) to very large, like the Internet and the Web have. Another key issue is providing reasonable infrastructure/framework support for building distributed applications. (After all, applications are what really drives the area.)

My broad theory is:

1. Application-specific shared state should be expressed in terms of "objects", which define type-safe interfaces.
2. Framework/infrastructure support should map these interfaces onto protocols that provide efficient sharing, minimizing manual involvement with the communication layer, while allowing application-level control over sufficient consistency.
3. Application logic should be implemented in terms of reading and writing these shared state objects, (largely) isolated from the distributed implementation of these objects.

In the following chapters, we explore this theory in detail, looking at the specifics of this framework support, the specifics of certain services and comparison and evaluation relative to conventional solutions and some systems. For the latter, we also look at papers giving examples of real system designs. The papers provide more details to complement the principles in these chapters. They also provide a contrasting viewpoint in some cases to the strong views running through this text.

[9]Folks in the distributed systems area orginally came largely from networking and operating systems, not counting theoreticians looking for problems to apply their techniques to a new area. The OS folks tend to focus on software rather than protocols.

Readings

M. Balazinska et al. Fault-tolerance in the Borealis Distributed Stream Processing System, SIGMOD 2005 June 14-16, Baltimore Maryland. (distributed streaming processing model and consistency/availability tradeoff)

References

D.Cheriton, The V Distributed System, CACM March 1988 (http://www.infosys.tuwien.ac.at/Staff/mj/dspapers/p314-cheriton.pdf)

M. Matijasevic, A Review of Networked Multi-user Virtual Environments, (http://citeseer.nj.nec.com/cache/papers/cs/2587/http:zSzzSzwww.acim.usl.eduzSz majazSztr_97-8-1.pdf/matijasevic97review.pdf)

J. Thorpe, The New Technology of Large-scale Simulator Networking: Implications for Mastering the Art of Warfighting, Proc. 9th Interservice/Industry Training Systems Conf. November 1992, p. 492-501.

B. Oki, M. Pfluegel, A. Siegel, D. Skeen, The Information Bus, SOSP93,p. 58-68.

A. Fox, et al. Cluster-based Scalable Network Services, SOSP'97, France, Oct 1997.

V. Cate and T. Gross, /alex: A Heterogeneous Network-wide Filesystem. September, 1992, available on /alex/edu/cmu/cs/sp/alex/doc.

D. Gifford et al. An Architecture of Large Scale Information Systems, SOSP'85